MainCrafts Internship

Artificial Intelligence & Machine Learning – Task 2

Feature Engineering, Model Optimization & Performance Comparison

Name :THANUJA M

Introduction

This task focuses on applying machine learning techniques to predict housing prices using regression models. The objective is to perform:

- Data preprocessing
- Feature engineering
- Feature scaling
- Model training
- Hyperparameter tuning
- Performance evaluation
- Model comparison

The dataset used for this task is the California Housing Dataset, which contains information about housing districts in California.

Objective of the Task

The main objectives of this task are:

- To understand regression modeling.
- To implement feature engineering techniques.
- To apply feature scaling.

Dataset Description

The California Housing Dataset contains the following features:

Feature Name	Description
MedInc	Median income in block

Feature Name	Description
HouseAge	Median house age
AveRooms	Average rooms per household
AveBedrms	Average bedrooms per household
Population	Block population
AveOccup	Average occupants per household
Latitude	Latitude
Longitude	Longitude

Target Variable:

- Median house value (housing price)

Feature Engineering

New features were created to improve model performance:

- Rooms_per_Household
- Bedrooms_per_Room
- Population_per_Household

These engineered features help capture relationships between existing features and improve predictive power.

```
# ==========================================
# 3. Feature Engineering
# ==========================================

# Creating new meaningful features
X["Rooms_per_Household"] = X["AveRooms"] / X["HouseAge"]
X["Bedrooms_per_Room"] = X["AveBedrms"] / X["AveRooms"]
X["Population_per_Household"] = X["Population"] / X["AveOccup"]

# Handle infinite values (if any)
X.replace([np.inf, -np.inf], np.nan, inplace=True)
X.fillna(X.mean(), inplace=True)

print("After Feature Engineering:", X.shape)

# ==========================================
# 4. Train-Test Split
# ==========================================

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Output

```
••• Dataset Shape: (20640, 8)
       MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
    0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
    1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
    2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
    3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
    4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

       Longitude
    0    -122.23
    1    -122.22
    2    -122.24
    3    -122.25
    4    -122.25
    After Feature Engineering: (20640, 11)
```

Data Preprocessing & Feature Scaling

Feature scaling was performed using **StandardScaler** to normalize the feature values.

Reason:

- Linear and Ridge Regression are sensitive to feature magnitude.
- Scaling improves convergence and performance.

```python
# ==========================================
# 6. Model 1 – Linear Regression
# ==========================================

lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

lr_pred = lr_model.predict(X_test_scaled)

lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
lr_r2 = r2_score(y_test, lr_pred)

print("\nLinear Regression")
print("RMSE:", lr_rmse)
print("R2 Score:", lr_r2)
```

Model Implementation

Three regression models were implemented:

Linear Regression

- Basic regression model
- No hyperparameter tuning

Output

```
Linear Regression
RMSE: 0.6958701266963986
R2 Score: 0.6304701499538703
```

Ridge Regression (Optimized)

- Used GridSearchCV
- Tuned hyperparameter: alpha

Output

```
Ridge Regression
Best Alpha: {'alpha': 10}
RMSE: 0.6959195832838997
R2 Score: 0.6304176219487071
```

Decision Tree Regressor (Optimized)

- **Tuned max_depth and min_samples_split**

```
Decision Tree Regressor
Best Parameters: {'max_depth': 10, 'min_samples_split': 10}
RMSE: 0.6290075326248504
R2 Score: 0.6980709971707264
```

Model Evaluation Metrics

The following metrics were used:

Root Mean Square Error (RMSE)

- Measures prediction error.
- Lower RMSE = Better model.

R^2 Score

- Measures how well data fits the model.
- Higher R^2 (closer to 1) = Better performance.

Model Comparison

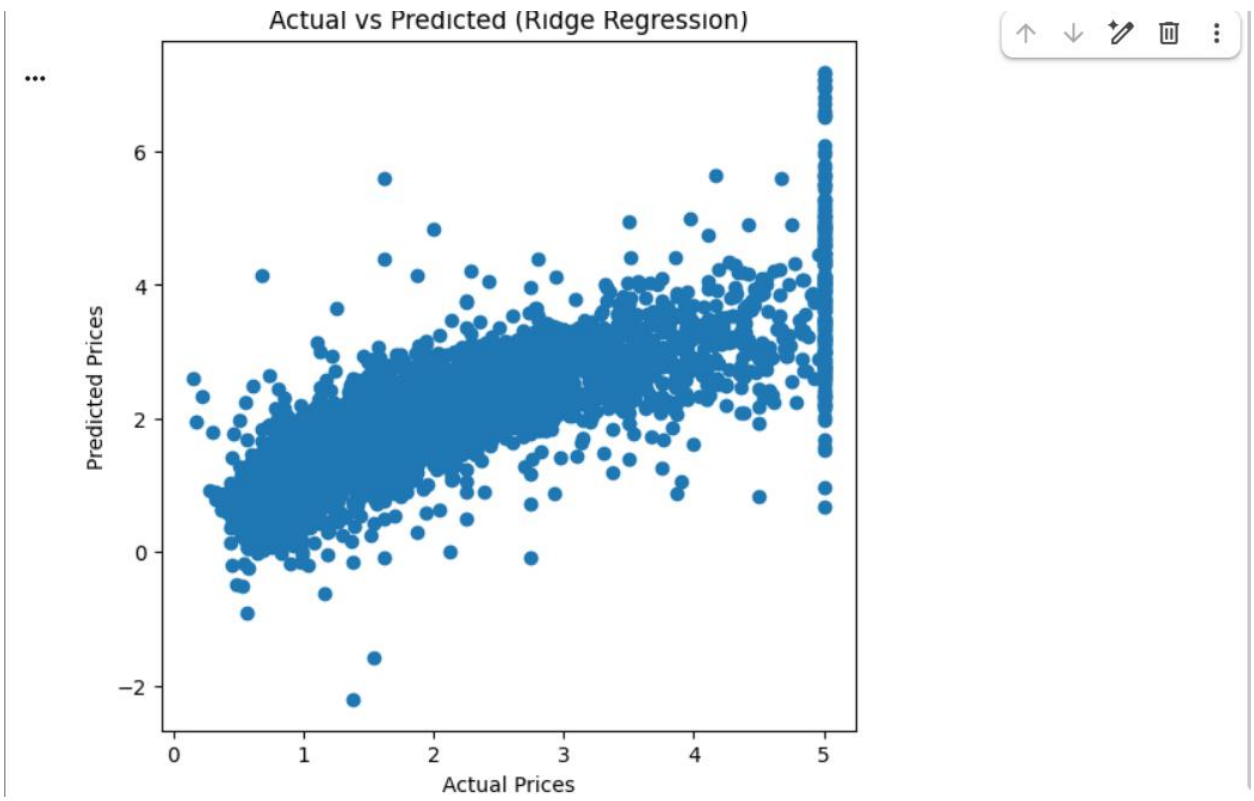
Model	RMSE	R^2 Score
Linear Regression	0.6958701266963986	0.6304701499538703
Ridge Regression	0.6959195832838997	0.6304176219487071
Decision Tree	0.6290075326248504	0.6980709971707264

Output

```
Model Comparison
                Model      RMSE  R2 Score
0  Linear Regression  0.695870  0.630470
1   Ridge Regression  0.695920  0.630418
2      Decision Tree  0.629008  0.698071
```

Visualization

An Actual vs Predicted scatter plot was generated to visualize model performance.



Final Conclusion

- Feature engineering improved model understanding.
- Ridge Regression performed better than basic Linear Regression.
- Decision Tree showed competitive performance but may risk overfitting.
- Hyperparameter tuning significantly improved results.